

POO 3

Tiago Henrique Angioletti





POLIMORFISMO

"Polimorfismo" em grego significa "muitas formas".

Traz flexibilidade e reutilização de código em C#



TIPOS DE POLIMORFISMO EM C#



POLIMORFISMO DE SOBRECARGA

Múltiplos métodos com o mesmo nome em uma classe, mas com parâmetros diferentes.

```
using System;
class Calculadora
{
    public int Somar (int a, int b)
    {
        return a + b;
    }

    public double Somar (double a, double b)
    {
        return a + b;
    }
}

class Program
{
    static void Main()
    {
        Calculadora calculadora = new Calculadora();
        Int resultadeinteiro = calculadora.Somar(5, 18);
        double resultadoDouble = calculadora.Soma(3.5, 2.5);
    }
}
```



POLIMORFISMO DE SUBSTITUIÇÃO

Subclasses podem ser tratadas como instâncias da classe pai.

```
using System;

// Classe base (pai)
class Animal
{
    public virtual void EmitirSom()
    {
        Console.WriteLine("0 animal emite um som genérico.");
    }
}

// Subclasse 1
class Cachorro : Animal
{
    public override void EmitirSom()
    {
        Console.WriteLine("0 cachorro late: Woof! Woof!");
    }
}

// Subclasse 2
class Gato : Animal
```



VANTAGENS DO POLIMORFISMO EM C#

Reutilização de Código

Flexibilidade e Extensibilidade

Manutenção Simplificada

Facilita o uso de interfaces e classes abstratas em C#



INTERFACES



CONTRATO:

Uma interface define um contrato que as classes que a implementam devem seguir. Isso significa que qualquer classe que implementa uma interface deve fornecer implementações para todos os métodos definidos na interface.



MÉTODOS SEM IMPLEMENTAÇÃO:

As interfaces contêm apenas a assinatura dos métodos, ou seja, os nomes dos métodos, seus tipos de parâmetros e seus tipos de retorno. Não há implementações de código real em uma interface.



FLEXIBILIDADE:

O uso de interfaces torna o código mais flexível e extensível, pois permite que você adicione novas funcionalidades a uma classe sem modificar sua implementação existente. Isso segue o princípio do "aberto/fechado" (open/closed) do design de software, que incentiva a extensão de código sem modificar o código existente.



PADRÕES DE DESIGN:

Interfaces são frequentemente usadas em conjunto com padrões de design, como o padrão Strategy, o padrão Observer e outros. Elas fornecem um meio eficaz de aplicar esses padrões de maneira consistente.

```
// Declaração da interface IAnimal
public interface IAnimal
{
    void FazerBarulho();
}
// Classe Cachorro que implementa a interface IAnimal
public class Cachorro : IAnimal
{
    public void FazerBarulho()
    {
        Console.WriteLine("0 cachorro late: Woof! Woof!");
    }
}
// Classe Gato que implementa a interface IAnimal
public class Gato : IAnimal
{
    public void FazerBarulho()
    {
        Console.WriteLine("0 gato mia: Meow!");
    }
}
```



HERANÇA MULTIPLA

Somente com interfaces

```
using System;
// Interface para veículos
public interface IVeiculo
{
    void Ligar();
    void Desligar();
}
// Interface para dispositivos eletrônicos
public interface IDispositivoEletronico
{
    void Ligado();
    void Desligado();
}
// Classe que implementa múltiplas interfaces
public class Carro: IVeiculo, IDispositivoEletronico
{
    public void Ligar()
    {
        Console.WriteLine("0 carro está ligado.");
    }
}
```